



Docker en siete comandos

Guía 1 · Taller 02 de caso aplicado de IA en industria

Todo lo que hay que saber de Docker para este taller cabe en una hoja. No es un curso de Docker: es la lista de lo que vas a teclear y qué hace cada cosa.

Cuatro palabras

PALABRA	QUÉ ES	LA PREGUNTA QUE RESPONDE
imagen	La receta congelada: sistema, librerías y código	¿Qué se va a ejecutar?
contenedor	Una ejecución de una imagen. Desechable	¿Dónde se está ejecutando?
volumen	Disco que sobrevive al contenedor	Si borro el contenedor, ¿dónde quedan mis datos?
servicio	Cada bloque de <code>docker-compose.yml</code> : <code>plataforma</code> , <code>agente</code> , <code>ui</code>	¿Cómo se llama la pieza?

Un modelo no es un contenedor. En este taller una sola imagen carga los 192 modelos. La relación no es uno a uno, y esa confusión es la que más cuesta soltar si vienes del taller anterior.

up contra run, que es la distinción que más se confunde

```
docker compose up -d ui          # levanta algo y lo DEJA VIVO escuchando
docker compose run --rm agente # ejecuta una tarea y BORRA el contenedor
```

`make ui` usa `up`. `make seed`, `make agente`, `make plano` y `make verificar` usan `run --rm`: de esos no queda ni rastro en `make estado`, y está bien que así sea — son tareas, no servicios.

Los siete comandos

```
make arriba      # construye las tres imágenes y levanta plataforma + interfaz
make seed        # crea el mundo: datos, entrena 192 modelos, pronostica, mide
make estado      # qué está corriendo ahora mismo
make ui          # abre la interfaz en http://localhost:8501
make abajo       # apaga todo, sin borrar nada
make logs        # sigue los logs. SERVICIO=agente para uno solo
make reset       # botón de pánico: borra TODO, incluidos los 192 modelos
```

`make ayuda` los lista todos. `cat Makefile` enseña qué hay detrás de cada uno, y detrás de cada uno hay un `docker compose` que puedes teclear entero.

Ojo con `make reset`. Es `docker compose down -v`, y esa `-v` borra los volúmenes. Después hay que volver a correr `make seed` — unos minutos. `make abajo` apaga sin borrar, que es lo que quieres casi siempre.

Dónde viven tus datos

En el volumen `datos`, montado en `/datos` dentro de los contenedores. **No en la carpeta del repositorio.** Por eso sobreviven a `make abajo` y por eso `make reset` los borra.

```
docker compose run --rm plataforma ls -la /datos
```

<code>ventas.csv</code>	76,800 filas	el mundo, dos años de historia
<code>modelos/</code>	192 <code>.joblib</code>	los artefactos entrenados
<code>predicciones.csv</code>	17,472 filas	lo que escribió el job anoche
<code>metricas.csv</code>	17,472 filas	lo único que el agente va a mirar
<code>ejecuciones_job.csv</code>		la bitácora del job

Si algo no arranca

«Cannot connect to the Docker daemon». Docker Desktop no está corriendo. Ábrelo, espera a que el icono deje de moverse, y reintenta.

Cambiaste un archivo y el contenedor sigue igual. Un contenedor sirve el código con el que arrancó. `docker compose restart plataforma`, o `make abajo && make arriba`. La pregunta que resuelve esto en cualquier laboratorio con volúmenes montados es: **¿el proceso vio este archivo?**

«No space left on device» a mitad de la construcción. Las imágenes ocupan unos 6 GB y la caché otro tanto. `docker builder prune -f`, y deja 12 GB libres antes de empezar.

MÓDULO III · Herramientas para la Inteligencia Artificial Generativa y aplicaciones para la industria
Taller 02 de caso aplicado de IA en industria